Replicability: Challenging the Myth of Randomness Patterns in Encrypted Traffic

Timofey Shpakov
timofey.shpakov@alumni.ethz.ch
ETH Zürich
Switzerland

Martin Burkhart
martin.burkhart@armasuisse.ch
armasuisse
Switzerland

Laurent Vanbever
lvanbever@ethz.ch
ETH Zürich
Switzerland

Abstract

Several recent studies claim that encrypted network traffic exhibits statistically significant randomness deficiencies due to differences in cryptographic algorithms, implementations, and parameterization, and that such patterns can be exploited for application classification in TLS traffic. This interpretation has been adopted by measurement researchers, most prominently in the often-cited ET-BERT study.

We present a systematic replication of these randomness experiments, re-implementing them using the NIST Statistical Test Suite on encrypted traffic generated under controlled conditions. In the replicated work, the suite is not itself a detection component, but is used to argue that detectable patterns exist in ciphertext and thereby motivate the method. It is this evidentiary step that we examine. Our evaluation shows that the reported statistical signals do *not* provide sufficient evidence of non-randomness beyond what is expected from statistical hypothesis testing and finite-sample effects. The observed test failures and discriminative patterns are consistent with methodological artifacts, including improper aggregation, p-value misinterpretation, and information leakage between training and test data.

Overall, our findings suggest that assumptions about imperfect randomness in encrypted network traffic are unsubstantiated. Our critique is limited to randomness-based classification, and we make no claim about methods exploiting metadata, packet sizes, timing, or other side-channel information.

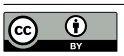
CCS Concepts

• **Networks** → **Network measurement**; • **Mathematics of computing** → *Hypothesis testing and confidence interval computation*; • **General and reference** → *Validation*; • **Security and privacy** → Web protocol security.

Keywords

Encrypted Traffic Analysis, Replication Study, Randomness Testing, Network Measurement, Statistical Evaluation, Transport Layer Security

ACM Reference Format:
Timofey Shpakov, Martin Burkhart, and Laurent Vanbever. 2026. Replicability: Challenging the Myth of Randomness Patterns in Encrypted Traffic. In *Proceedings of the 2026 ACM Internet Measurement Conference (IMC '26), October 12–16, 2026, Karlsruhe, Germany*. ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3777912.3839783



This work is licensed under a Creative Commons Attribution 4.0 International License.
IMC '26, Karlsruhe, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2327-8/26/10
https://doi.org/10.1145/3777912.3839783

1 Introduction

Today, more than 95% of all Internet traffic is encrypted, according to data reported by Cloudflare [10]. While this benefits the security of users, the problem of "going dark" [14] complicates network management tasks such as IDS/IPS, malware detection, application classification, QoS, and performance monitoring. In response, methods have focused on differentiating encrypted traffic using information that remains observable in plaintext, behavioral characteristics, and auxiliary metadata sources [13, 26]. These features include strong identification information (SII) such as MAC and IP addresses, protocol ports, and certificate data, as well as traffic flow characteristics [4, 38] and side-channel metadata such as packet sizes [27, 30] and timing patterns [12]. While these approaches partially restore visibility into encrypted traffic, they rely on information that is increasingly targeted by protocol-level countermeasures that limit information leakage. Examples include DNS queries performed over TLS or HTTPS (DoT and DoH) and the Encrypted ClientHello (ECH) extension in TLS 1.3, developments that progressively reduce the protocol metadata accessible to passive observers and narrow the scope of viable traffic analysis techniques.

More recently, deep learning approaches, such as Deep Packet [23], ET-BERT [22], App-net [39], YaTC [43], and others [1, 9, 32, 42], started training classifiers on the raw, encrypted bytes of network traffic, hoping to capture statistical patterns caused by differences in cryptographic algorithms, implementation details, and parameterization. Sengupta et al. [31] explicitly tested for such patterns and concluded that *"the tell-tale differences in randomness [...] begin emerging strongly when the sequence is transformed to the frequency domain (using Fast Fourier Transform)"*. This interpretation has been adopted by some measurement researchers, most prominently in the often-cited ET-BERT study [22]. The authors motivate their approach by attributing ET-BERT's high classification performance largely to application-specific randomness patterns. They reproduce the underlying randomness experiments of Sengupta et al. and adopt their conclusion that significant randomness patterns do exist in encrypted traffic.

If encryption actually produced significant randomness patterns, work adhering to the current consensus of solely relying on traffic metadata would miss out on a great opportunity and the community would be well advised to adjust its best practices. Yet, this would also raise serious concerns regarding the security of modern ciphers. The ciphertext indistinguishability assumption (IND-CPA) is the foundation of semantic security [7], underlying virtually every

protocol protecting confidentiality in the Internet today. If it is possible to distinguish from randomness alone whether AES-GCM or ChaCha20 was used, or infer any information about the cleartext – e.g., if an image or a text document is encrypted – the encryption scheme fails to be semantically secure.

Therefore, we put the claim of significant randomness patterns in encrypted network traffic to the test, thoroughly replicating the randomness experiments of Sengupta et al. [31] and Kanda and Hashimoto [17]. We closely adhere to the experimental designs of the original studies to assess whether their findings can be independently confirmed. Specifically, we re-implement the experiments using the NIST Statistical Test Suite on encrypted traffic generated under controlled conditions, discussing methodological subtleties that are often omitted. The replicated studies [17, 31] evaluate this premise on cipher classification. That task is not an end in itself, but it is a necessary condition for the premise to hold: ciphertexts produced by different ciphers can only be told apart if the ciphers leave distinguishable patterns behind. It is also the easiest place to look for such patterns, with few classes, a direct hypothesis, and full control over the generating process. If they do not appear here, the more ambitious tasks named above lose the randomness-based justification on which they were motivated.

Scope. We do not challenge the long line of work analyzing encrypted traffic through the metadata side channels described above, such as packet sizes, timing, flow-level features, and TLS handshake metadata. What we challenge is the distinct and more recent claim that the statistical properties of the encrypted payload bytes themselves contain exploitable structure.

Our evaluation shows that the reported statistical signals do *not* provide sufficient evidence of non-randomness beyond what is expected from well-known properties of statistical hypothesis testing and finite-sample effects. The observed test failures and discriminative patterns are consistent with methodological artifacts, including improper aggregation of test outcomes, misinterpretation of p-values, and experimental setups that induce information leakage between training and test data.

Overall, our findings suggest that assumptions about imperfect randomness in encrypted network traffic are unsubstantiated. Our study underscores the necessity of rigorous statistical reasoning and independent validation in network measurement research, as many others have done before us [5, 16, 19, 34, 37, 40, 44].

2 Related Work

Methodological rigor in network measurement experiments has a long tradition of scrutiny. Kim et al. [19] set an early precedent for this type of methodological "myth-busting", Sommer and Paxson [34] caution against mistaking dataset artifacts for intrusion signals, Juarez et al. [16] show that impressive lab accuracy in website fingerprinting may fail in practice due to improper evaluation assumptions, and Bajpai et al. [5] address reproducibility challenges caused by incomplete documentation, environment sensitivity, and unstated assumptions.

Wickramasinghe et al. [40] provide a systematic critique of prominent approaches, including ET-BERT and YaTC [43], and show that reported performance is frequently attributable to confounding factors rather than ciphertext properties. These factors include residual plaintext artifacts in datasets, leakage through packet sizes or protocol framing, and experimental design issues such as improper splitting that induces leakage between training and evaluation. Zhao et al. [44] reach a similar conclusion for representation learning models, where reported performance is driven by shortcuts introduced during data preparation rather than by the traffic representation itself. These studies identify what such detectors do rely on, working top-down from the models. They leave open the prior question of whether randomness patterns exist in encrypted payloads at all and could be exploited by some detector.

We ask and answer that question directly, working bottom-up from the statistical tests. Our contribution is a statistics-level dissection of the randomness-testing evidence in Sengupta et al. [31] and Kanda and Hashimoto [17], together with the analytical length-proxy result in Section 6. The two directions are complementary, as a top-down result is bounded by the model it examines, whereas the bottom-up result applies to any method that would rely on the same premise. That premise is shared by [17], [31], and [22]: that the NIST STS detect something in ciphertext, and that this implies exploitable hidden patterns exist. In [31] and [22] the suite is not part of the detection pipeline at all, but is used to argue that such patterns exist and thereby to motivate the method. In [17] features are instead derived from NIST STS statistics, and we show how side-channel information enters through those features. In neither case do we claim that the reported performance is wrong. For ET-BERT [22] in particular, if the statistical basis does not hold, the stated reason why a transformer should find anything useful in ciphertext bytes falls away, leaving side-channel information of the kind demonstrated by Wickramasinghe et al. [40] as the remaining explanation.

3 Methodology

This section outlines the methodology used to evaluate the randomness properties of encrypted data in offline and network settings. Both datasets are generated in a way that is controlled and aligned with recommended practices in statistical randomness testing. Since standard batteries such as NIST SP 800-22 [6] are specified for specific input conditions and are often used in the literature in a simplified or outdated manner, we document the necessary adaptations and evaluation choices made in this study. We also document reproduction details for two papers in the area of randomness-based classification, as key experimental parameters are not fully specified in the original works.

Dieharder and TestU01. We use the NIST STS to replicate the original studies exactly and to focus on proper test methodology rather than testing power. Other batteries such as Diehard [25], Dieharder [8] and TestU01 [20] overlap substantially with the NIST STS, and we discuss in Appendix D why repeating the experiments with them would not change our conclusions.

3.1 Dataset Generation and Collection

Dataset collection and generation is a key aspect often kept short in the literature in the area of randomness-based classification and cipher classification. Moreover, in cases such as Lin et al. [22] and Sengupta et al. [31], the datasets used for statistical analysis are not even suitable for a sound evaluation, as too few samples are

generated and recommended practices in randomness testing are not followed. In contrast to much of the literature dealing with cipher classification, our aim is to study a realistic setting that could be found in real network traffic today. For example, we do not aim to find patterns induced by legacy ciphers operating in Electronic Codebook (ECB) mode.

We generate new datasets rather than reusing those of the replicated studies. This is a replication study, and replication does not in general require the original data. Moreover, the datasets in both [31] and [17] were generated by the authors and are not publicly available. We were in brief contact with the authors of [31] to clarify the data generation procedure for the NIST STS experiments, but did not obtain code or a clarification. Generating our own data also gives us full control over the conditions under which the statistical claims are tested, which is what allows us to construct the deliberately adversarial settings used in Section 4.

3.1.1 Offline Data. As a starting point, we examine the standard encryption provided by the widely used Python library `PyCryptodome`. The data generation process is adapted from the work of Sengupta et al. [31], who investigated differences in TLS implementations across various Android applications. Their study is used as a reference point, as it serves as the basis for the reproduction attempts described in later subsections.

The dataset is constructed by generating m plaintext sessions, where each session consists of n packets of size b bytes. For simplicity and to potentially increase the visibility of induced patterns, we initially use constant plaintext consisting of zero bytes. To keep terminology consistent across offline and network experiments, we refer to each generated plaintext session as a session and each block of b bytes as a packet. This offline usage of session and packet denotes fixed-size blocks and their groupings, and does not refer to network packets or TLS sessions in the usual sense. To keep offline data generation reproducible, `PyCryptodome`'s `get_random_bytes` is replaced by a deterministic generator based on Python's `random` module with a fixed seed. For each session, we pre-generate all cryptographic material and keep it consistent across ciphers for the same plaintext session. Concretely, we use one session key per session and generate per-packet IVs or nonces to avoid reuse within a session. The evaluated ciphers and parameters are listed in Table 1. The key sizes in the table are not necessarily the only supported ones, and we use 16-byte keys where possible due to broad support across ciphers. To keep keys as consistent as possible, we generate a 32-byte key per session and truncate it according to the cipher requirements. We generate per-packet IVs and nonces analogously.

PKCS#1 refers to the asymmetric RSA cipher, which can operate with either version 1.5 padding or Optimal Asymmetric Encryption Padding (OAEP). Since public-key cryptography is not designed for bulk encryption, it limits the plaintext size, so asymmetric ciphers are used only in experiments where the packet size permits. The pre-generated cryptographic material is employed to encrypt sessions, with the session key rotated per session while IVs or nonces are used per packet within a session. Block ciphers are operated in CBC mode with explicit initialization vectors, meaning that each packet uses a separate IV rather than chaining across packets.

Cipher	Type	Key [Bytes]	IV [Bytes]	Nonce [Bytes]
AES	Block	16	16	-
Blowfish	Block	16	8	-
CAST	Block	16	8	-
DES	Block	8	8	-
DES3	Block	16	8	-
ARC2	Block	16	8	-
ChaCha20	Stream	32	-	8
ARC4	Stream	16	-	-
Salsa20	Stream	16	-	8
XOR	Stream	16	-	-
PKCS#1 v1.5 (RSA)	Asymmetric	256	-	-
PKCS#1 OAEP (RSA)	Asymmetric	256	-	-

Table 1: Overview of the evaluated generators and encryption parameters.

Additionally, we generate sessions using the PRNG Mersenne Twister and using a large blob of QRNG (Quantum Random Number Generator) data. The QRNG data is provided by the Quantum Optics Group of the Australian National University (ANU), with its generation described by Haw et al. [15]. The original hosting was discontinued, and we obtained a copy from Aqacia, an ANU spin-off. We create sessions from QRNG data by cutting the blob into streams of size $n \cdot b$.

3.1.2 Network Data. Encrypted payloads in network traffic differ from data encrypted offline in various aspects. In the offline setting, the input to encryption is raw bytes, while in the network setting an application protocol is carried and protected, which introduces record framing overhead and integrity protection. In TLS 1.2 with CBC suites, integrity is provided via HMAC in a MAC-then-encrypt construction, while in AEAD suites the integrity tag is integrated into the encryption scheme, so the resulting ciphertext length can differ even for identical application data depending on the negotiated algorithms and record formatting. Furthermore, TLS 1.3 encrypts most handshake messages after the ServerHello, which reduces handshake visibility compared to TLS 1.2 and increases the fraction of encrypted bytes observed in captures [29]. Both effects are what the length analysis in Section 6.2 relies on.

Apart from the cryptographic aspects, a major challenge is reproducibility. In the case of offline encryption, one can switch from cryptographically secure PRNGs to a deterministic PRNG. To guarantee reproducibility, drastic modifications to client and server code and configuration are required that push the lab setting further away from real network traffic. Thus, we prioritize realistic protocol behavior over full determinism and document all software versions, configurations, and payload corpora to make the setup reproducible at the level of the experimental environment. This virtualized range is intended to replicate the lab conditions of the original studies under controlled conditions, not to demonstrate that any method generalizes to real-world traffic. Establishing generalization would require measurement on real Internet-connected devices, which is outside the scope of this replication.

Now, to generate the simulated network data, a TLS traffic range is implemented which creates web server instances for specified TLS versions and cipher suites. The range consists of containerized web servers and a collector container, all connected to an isolated private

network, as illustrated in Figure 1. Each server is configured to fix the TLS version and cipher suite, so that every capture is attributable to exactly one configuration. The collector supports TLS 1.0 to TLS 1.3. We pre-generate a fixed 1504-byte blob of `0x00` bytes, mounted into the web roots. Using `tcpdump` and scripts executed from the collector container, we capture traffic and write session keys to `SSLKEYLOGFILE`, which allows decrypting captures for validation and for extracting payload-level artifacts. Full deployment details and software versions are given in Appendix C.

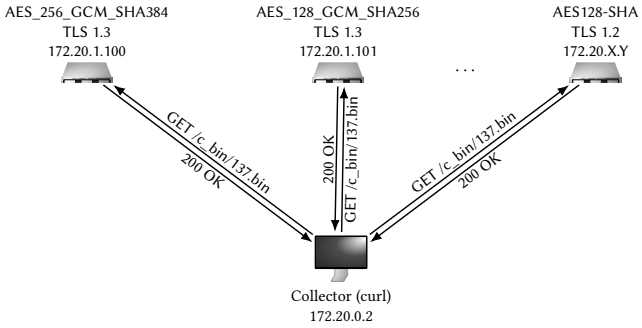


Figure 1: Illustration of the TLS range consisting of web servers and a collector container in an isolated private network.

We compare TLS 1.2 and TLS 1.3 in Section 6.2, where the protocols are shown to leak information about protocol choice through session length, independently of any question of insufficient randomness.

3.2 Randomness Testing

The section outlines the methodology used to test the randomness of ciphertext produced by different ciphers. Two complementary approaches are applied: a statistical approach based on established randomness test suites, and an algorithmic approach using compression-derived features. The following subsections describe the selected implementations, modifications made for practical use, and the feature extraction process.

3.2.1 Statistical. This subsection motivates the choice of statistical test suite implementation. For NIST SP 800-22 [6], multiple implementations exist. A notable Python implementation is `r4nd0m` by Stuart Reid [28], which was also used by Sengupta et al. [31]. NIST additionally provides a reference C implementation of the STS. After evaluating both, we chose the NIST reference implementation as the baseline to minimize ambiguity with respect to the standard and to ensure that the executed tests match the specification as closely as possible. In preliminary comparisons, we observed implementation-dependent differences in returned p-values for some tests, which further motivated this choice. The modifications required to run the suite in an automated pipeline, together with two robustness issues we patched, are documented in Appendix E.

Because it features prominently in the work we replicate, we briefly describe the DFT-based spectral test. The bit sequence is mapped to ±1 and transformed with a discrete Fourier transform, and the magnitudes of the resulting components are compared against a threshold below which 95% of them should fall under the null hypothesis of randomness. A periodic pattern in the sequence concentrates energy at the corresponding frequency, producing more components above the threshold than expected. The test statistic is the normalized difference between the observed and expected counts of such components, from which a p-value is derived [6]. The test therefore detects periodic or repetitive structure that tests operating on bit frequencies or run lengths may not reveal, which is the property Sengupta et al. [31] invoke when motivating frequency-domain features.

3.2.2 Algorithmic. As a complementary proxy for algorithmic randomness, we derive compression-ratio features from general-purpose lossless compressors, motivated by the interpretation of Kolmogorov complexity as the length of the shortest description of a sequence and its relation to compressibility [21]. Intuitively, highly random sequences should remain incompressible, while structured generators may exhibit redundancy that compressors can exploit. The feature construction, the set of compressors used, and the results of compression-based randomness detection are provided in Appendix G.

4 Randomness Testing of Ciphers

Before randomness-based classification of encrypted traffic can be considered credible, it is necessary to establish that ciphertext produced by different ciphers forms distinguishable populations under sound evaluation practices. We therefore start with an offline experiment in which encryption inputs and key material are controlled, allowing us to test for cipher-dependent effects in the absence of protocol artifacts. We then contrast the cipher results with deliberately non-random generators to calibrate the sensitivity and limitations of the test methodology. Finally, we repeat the analysis in a network setting using TLS-protected traffic, where ciphertext is embedded in real protocol records and therefore subject to framing, integrity overhead, and implementation effects.

4.1 Offline Data

In a first experiment we create deliberately adversarial conditions for detecting cipher-dependent artifacts. Using the setup from Subsection 3.1.1, we generate 1000 sessions, each consisting of 1000 packets of 128 bits. The plaintext is a constant stream of zero bytes to maximize any deterministic structure that could survive encryption. Within each session, packets are concatenated before testing to expose potential patterns across multiple ciphertext blocks produced under the same key and cipher state. We evaluate AES, ARC2, ARC4, Blowfish, CAST, DES, Triple DES, PKCS#1-OAEP, PKCS#1-v1.5, XOR, a QRNG source, the Mersenne Twister, and the raw plaintext.

We perform statistical randomness testing on the generated dataset as described in Subsection 3.2.1 using the NIST Statistical Test Suite (NIST STS) [6]. Figure 2 shows a violin plot of p-values from the spectral test across generators.

The spectral test is highlighted because Sengupta et al. [31] motivate their feature engineering using frequency-domain behavior and describe the DFT-based test as particularly discriminative. In our setting, the cipher outputs exhibit broadly similar p-value distributions with no stable, visually separable signature per cipher.

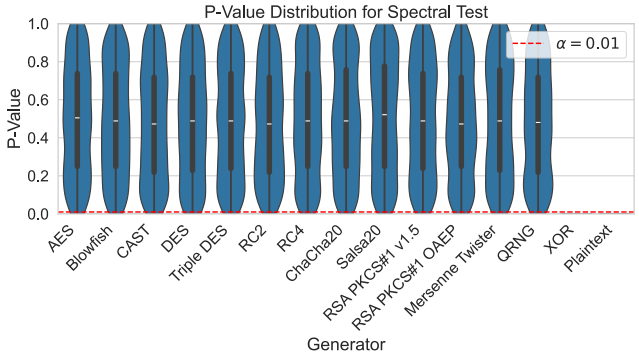


Figure 2: Violin plot of the spectral test p-values over different generators.

As expected, the Plaintext and XOR baselines fail as the plaintext consists entirely of zeros, and XOR reduces to a repeating key pattern when applied to a zero plaintext. Across the remaining NIST STS tests, the qualitative picture is the same with cryptographic ciphers behaving like high-entropy sources.

To calibrate the sensitivity of the NIST STS [6] beyond the trivial pass and fail extremes, we additionally evaluate four non-cipher generators: structured, Bzip2, Gzip, and XZ. The structured generator is formed by concatenating NIST publications in PDF format. The remaining generators are the corresponding compressor outputs. Since the available corpus is limited, sequence-generation parameters are adapted accordingly. In particular, XZ yields the smallest data blob due to its compression efficiency, which limits us to generating 100 sequences consisting of 1000 blocks of size 1024 bits.

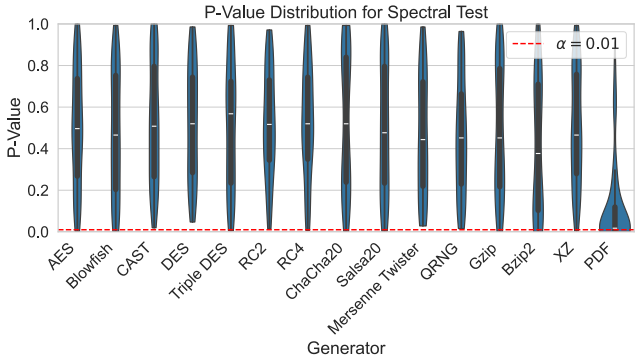


Figure 3: Violin plot of the spectral test p-values over different generators including known non-random generators.

The results in Figure 3 show that the spectral test clearly detects non-random structure in the structured and Bzip2 generators, reflected by a larger mass of p-values near 0. Considering another test, such as the runs test in Figure 4, also reveals that Gzip is detectably non-random under this battery. In contrast, the p-value distributions of XZ resemble those of the cipher outputs, the Mersenne Twister, and the QRNG, with individual test failures remaining within the rate expected at $\alpha = 0.01$.

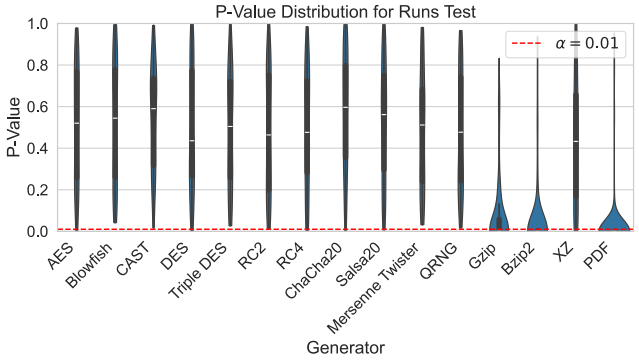


Figure 4: Violin plot of the runs test p-values over different generators including known non-random generators.

However, when examining the NIST-recommended proportion of passing sequences alongside p-value uniformity, XZ behaves differently. Table 2 shows that several tests have pass proportions that deviate noticeably from the nominal rate of $1 - \alpha = 0.99$ implied by the significance level, even when the corresponding p-value distributions do not stand out visually.

Overall, two observations follow from the offline experiments. First, all evaluated cryptographic ciphers have no difficulty passing the NIST STS under this setting, visible as the predominantly green cipher columns in Table 2. Occasional failures of individual tests occur (isolated red cells), but NIST SP 800-22 explicitly warns against treating a small number of failed subtests as decisive, and recommends re-evaluation on higher volume when results are borderline [6]. Second, the suite is sensitive enough to flag some seemingly "high entropy" non-cipher sources, e.g. structured data and common compressors, while also showing that certain compressor outputs can be difficult to distinguish from cipher outputs using within-sequence statistical tests alone.

Before moving on to network data, it is important to highlight a blind spot of standard test batteries. They primarily test within-sequence properties under an IID assumption and therefore do not target inter-sequence structure. As an example, ARC4 can achieve good NIST STS performance even though its second keystream byte is known to be biased towards zero [24]. A practical mitigation when probing for these effects is to design sequences that aggregate across many independent keys, rather than treating each long sequence as an isolated IID sample.

4.2 Network Data

Next, we repeat the same statistical tests in a network setting, where ciphertext is embedded in a protocol stack and affected by record framing, integrity protection, and handshake artifacts.

Network traffic is generated using the TLS range described in Subsection 3.1.2. For each configured web server, the collector retrieves 1000 HTTP responses containing a fixed 1504-byte blob of `0x00` bytes, yielding 1000 samples per cipher suite. The all-zero payload is a deliberate choice. Since the claim under test is that different ciphers produce different statistical properties, the payload must not contribute any randomness of its own, and using only `0x00`

Ciphers / Tests	AES	ARC2	ARC4	Blow-fish	Bzip2	CAST	Cha-Cha20	DES	DES3	Gzip	Mer-senne	Plain-text	QRNG	Salsa20	Struc-tured	XOR	XZ
Approximate Entropy	1.00	1.00	1.00	0.99	0.18	0.98	1.00	1.00	0.99	0.23	0.99	0.00	0.99	0.99	0.00	0.00	0.96
Block Frequency	0.99	1.00	0.99	1.00	0.16	1.00	1.00	1.00	1.00	0.00	0.99	0.00	0.99	1.00	0.00	0.67	0.96
Cumulative Sums	0.99	1.00	0.99	0.99	0.37	0.98	0.99	0.98	0.99	0.39	1.00	0.00	0.98	1.00	0.03	0.07	0.96
Spectral	0.99	1.00	0.99	0.99	0.89	1.00	0.98	1.00	0.99	0.97	1.00	0.00	1.00	0.98	0.55	0.00	0.98
Frequency	0.99	0.99	0.99	0.98	0.44	0.98	0.98	0.98	0.99	0.41	1.00	0.00	0.98	1.00	0.09	0.07	0.96
Linear Complexity	0.99	0.99	1.00	1.00	0.99	1.00	0.98	0.98	0.96	0.98	0.98	0.00	0.99	1.00	0.68	0.00	1.00
Longest Run	0.99	0.99	0.97	0.99	0.40	0.99	1.00	1.00	0.98	0.96	1.00	0.00	0.97	0.97	0.50	0.00	0.95
Non-Overlapping Template	0.99	0.99	0.99	0.99	0.85	0.99	0.99	0.99	0.99	0.92	0.99	0.00	0.99	0.99	0.43	0.00	0.98
Overlapping Template	0.99	0.99	0.98	0.99	0.48	1.00	0.99	0.94	0.99	0.77	0.98	0.00	0.98	0.98	0.05	0.00	0.97
Random Excursions	0.99	0.99	0.99	0.99	0.98	0.99	0.99	0.98	0.99	1.00	1.00	-	0.99	0.98	-	-	0.99
Random Excursions Variant	1.00	0.99	1.00	0.99	0.97	1.00	1.00	0.98	1.00	1.00	1.00	-	0.99	0.99	-	-	1.00
Rank	0.99	0.99	0.98	1.00	0.99	1.00	1.00	0.99	1.00	1.00	0.97	0.00	1.00	0.99	0.22	0.00	0.99
Runs	0.99	1.00	0.98	1.00	0.20	1.00	0.99	0.99	1.00	0.35	1.00	0.00	1.00	0.99	0.05	0.02	0.95
Serial	0.99	1.00	0.99	1.00	0.38	0.98	0.98	0.99	1.00	0.59	0.99	0.00	1.00	0.97	0.00	0.00	0.96
Universal	1.00	0.99	1.00	0.98	0.51	0.99	1.00	0.99	1.00	0.11	0.99	0.00	0.99	0.97	0.00	0.01	0.94

Table 2: Proportion of tests passed for all tests for various ciphers / generators. Green cells meet the acceptable proportion criterion at $\alpha = 0.01$, red cells fall below it. Following NIST SP 800-22, the criterion is the lower bound $0.99 - 3\sqrt{0.99 \cdot 0.01/m}$, giving 0.9806 for the generators evaluated on $m = 1000$ sequences and 0.9602 for Bzip2, Gzip, Structured, and XZ, which are limited to $m = 100$. Displayed values are rounded to two decimals, while pass / fail is determined on the unrounded proportions.

Cipher Suite	Length [Bytes]
ECDHE-RSA-AES256-GCM-SHA384	1782
DHE-RSA-AES256-GCM-SHA384	1782
ECDHE-RSA-CHACHA20-POLY1305	1774
DHE-RSA-CHACHA20-POLY1305	1774
ECDHE-RSA-AES128-GCM-SHA256	1782
DHE-RSA-AES128-GCM-SHA256	1782
ECDHE-RSA-AES256-SHA384	1774
DHE-RSA-AES256-SHA256	1808
ECDHE-RSA-AES128-SHA256	1808
DHE-RSA-AES128-SHA256	1808
ECDHE-RSA-AES256-SHA	1796
DHE-RSA-AES256-SHA	1796
ECDHE-RSA-AES128-SHA	1796
DHE-RSA-AES128-SHA	1796
AES256-GCM-SHA384	1782
AES128-GCM-SHA256	1782
AES256-SHA256	1808
AES128-SHA256	1808
AES256-SHA	1796
AES128-SHA	1796

Table 3: Response packet length over different cipher suites.

bytes ensures that any statistical structure we observe can originate only from the encryption. This is the most adversarial condition under which to look for cipher-specific patterns. We restrict this experiment to TLS 1.2 and the cipher suites listed in Table 3. Since TLS 1.2 cipher suites differ in their integrity mechanism, e.g. AEAD vs. MAC-then-encrypt, and handshake or key-exchange overhead, the observed TLS records and resulting TCP segments may differ in length even when the application payload is fixed.

Table 3 reports the observed response packet lengths for a fixed 1504-byte application payload. As expected, suites that differ only in the key-exchange method, e.g. ECDHE vs. DHE, yield identical lengths in this measurement, while MAC-then-encrypt CBC suites and AEAD suites exhibit different overheads. To remove length as a trivial discriminator and focus the analysis on within-sequence bit statistics, we truncate each extracted byte sequence to 1504 bytes before converting to bit strings for NIST STS. Note that these 1504 bytes are taken from the encrypted application-data region observed on the wire and therefore still include the surrounding HTTP bytes encapsulated inside TLS. The truncation ensures that all compared inputs to the statistical tests have identical length. The value of 1504 bytes is inherited from Sengupta et al. [31] for comparability, and no part of our result depends on this specific value. Since the tested sequence is the extracted TLS application data rather than individual wire segments, it is independent of how the traffic was segmented on the wire and is not constrained by the MSS.

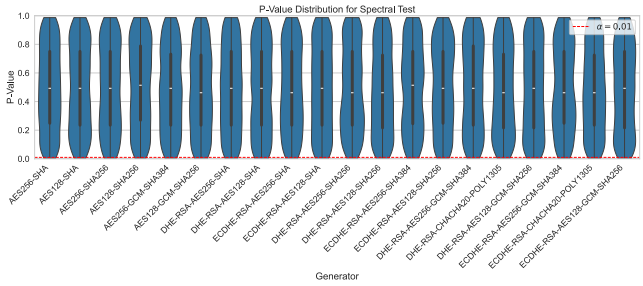


Figure 5: Violin plot of the spectral test p-values over different cipher suites.

Figure 5 shows the p-value distributions of the NIST STS spectral test across the evaluated TLS 1.2 cipher suites. The outcome mirrors the offline setting as the p-value distributions remain largely indistinguishable across suites, and we do not observe stable, suite-specific deviations that would support the hypothesis of cipher-dependent "randomness fingerprints" in bulk TLS application data under this workload.

These results motivate a closer examination of how prior work operationalizes randomness testing and feature extraction in practice, and to what extent reported classification performance can instead be explained by confounding variables such as length, protocol artifacts, or data leakage. We therefore turn to a reproduction-based analysis of Sengupta et al. [31] and Kanda and Hashimoto [17]. Note that we are focusing on their randomness experiments only, without replicating the detection aspects of the studies.

5 Reproducing Sengupta et al. (2019)

Sengupta et al. [31] report that bit-sequence features derived from encrypted payloads significantly improve encrypted traffic classification compared to timing- and packet-level baselines. A key motivation is the claim that some NIST STS tests, and in particular the spectral test, exhibit higher variance in p-values across ciphertexts produced by different ciphers, suggesting cipher-specific statistical fingerprints, summarized in Table 4. To support this, they invoke prior work on cryptographic randomness testing [3, 11, 18, 35], which they interpret as evidence that different ciphers produce outputs with measurably distinct statistical properties. However, these tests produce a qualitative pass/fail verdict on whether a cipher behaves sufficiently like a random mapping to be fit for use, and are not designed to quantify or compare the degree of randomness across ciphers that pass. At most, those results distinguish weak from strong entropy generators, a distinction irrelevant to TLS, where only the latter category is deployed. Building on this misinterpretation, the authors derive frequency-domain features from block- and packet-level bit sequences and report strong performance in distinguishing ciphers and key sizes and in classifying encrypted traffic from 175 Android applications [31].

ET-BERT [22] builds on these results and argues that its strong classification results are due to application-specific randomness patterns. The authors of ET-BERT replicated Sengupta's randomness experiments, but the methodology is not critically examined. Consequently, the NIST STS argument of Sengupta et al. [31] is inherited by ET-BERT.

The following reproduction study revisits the randomness-testing evidence used to justify cipher distinguishability and the subsequent DFT-based feature engineering pipeline.

5.1 Randomness Testing of Ciphers

5.1.1 Sengupta et al.'s Randomness Testing Methodology. In Sengupta et al. [31], the authors construct test sequences by generating 1000 data blocks of 1504 bytes using `SystemRandom` from Python's `random` module, backed by `/dev/urandom`, and encrypting them with multiple ciphers (AES, ARC2, ARC4, Blowfish, CAST, DES, Triple DES, PKCS#1 OAEP RSA, PKCS#1 v1.5 RSA, XOR, and an unencrypted baseline) [31]. The block length is chosen to match a typical MTU-sized payload. For symmetric primitives, a fixed 128-bit key and CBC mode are reported, while for public-key encryption the authors generate a 1024-bit RSA key pair via OpenSSL and encrypt blocks with the public key [31]. Randomness testing of the NIST STS [6] is performed using the unofficial Python implementation `r4nd0m` by Reid [28].

Several implementation details that materially affect reproducibility are not fully specified in [31]. First, it is unclear whether the 1000 blocks are evaluated as 1000 independent sequences or concatenated into a smaller number of long sequences prior to applying NIST STS tests, which matters because some tests require long inputs, e.g. the universal test, and because NIST STS is typically interpreted over a set of sequences rather than a single output stream [6]. Second, IV generation for CBC encryption is not described. Depending on whether IVs are fixed, random per block, or random per session, the injected randomness can dominate statistics at the tested lengths. Finally, the reported behavior of the unencrypted baseline in Table 4, where several tests yield p-values of 0 despite the plaintext being generated from `/dev/urandom`, is difficult to reconcile with standard interpretations of p-values under the null hypothesis and motivates a careful re-evaluation of the experimental pipeline.

More fundamentally, the randomness-testing evidence in [31] is difficult to interpret under NIST's recommended evaluation procedure. *NIST SP 800-22 emphasizes that conclusions should not be drawn from a single p-value.* Instead, one evaluates the proportion of sequences passing each test at a chosen significance level and the uniformity of the resulting p-value distribution across sequences [6]. These are two distinct criteria. The proportion criterion asks what fraction of sequences pass the test, i.e., have a p-value at or above the significance level, and is expected to approximate $1 - \alpha$ for a random source. The uniformity criterion asks whether the p-values themselves are spread evenly over $[0, 1)$, which is what the null hypothesis predicts. A generator can satisfy one and fail the other, so both are required before a source is accepted.

This distinction also explains the p-values reported per cipher in Table 4. Under the null hypothesis, valid p-values are uniformly distributed on $[0, 1)$, so deviations from uniformity are themselves evidence of non-randomness rather than a discriminative "signature" of a cipher. What converges under repetition is the distribution of p-values over many sequences, not any individual p-value. Based on the description in [31], each reported p-value is computed on a single long sequence per cipher, which gives an effective sample size of one. Assuming the ciphers are sufficiently random, each such value is therefore a single draw from the uniform distribution, and the differences between them across ciphers carry no information. For the same reason, a single draw admits no meaningful confidence interval, and the spread of values in Table 4 is not a dataset size effect but the expected behavior of the test. Additional practical guidance for interpreting NIST STS outcomes, including caveats and revised thresholds, is provided by Sýs et al. [36]. In the remainder of this section, we therefore reproduce the experiment using a multi-sequence evaluation and report both proportion and uniformity criteria.

5.1.2 Our Randomness Tests. Following [6], we reproduce the cipher-randomness experiment of Sengupta et al. [31] under a setting that intentionally maximizes any exploitable structure in the plaintext. Concretely, we generate plaintext blocks of 1504 bytes consisting entirely of `0x00` and encrypt them using the same packet and session abstraction introduced in Subsection 3.1.1. We evaluate AES, DES, Triple DES, Blowfish, ARC2, CAST, ChaCha20, ARC4, Salsa20, XOR, a QRNG source, the Mersenne Twister PRNG, and an unencrypted baseline. As in our offline experiments, we refer to one 1504-byte block as a packet and to an ordered sequence of packets encrypted under the same key as a session. For each cipher, we generate 1000 sessions where within each session, packets are encrypted in CBC mode using a fixed session key. We exclude RSA-based encryption from this reproduction because 1504-byte payloads cannot be encrypted directly with a 1024-bit RSA public key without chunking and additional padding logic, which would depart substantially from the original setup.

5.1.3 Discussion. Across all evaluated ciphers, excluding the deliberately non-random baselines, the Spectral Test p-values shown

Ciphers / Tests	AES	ARC2	ARC4	Blow-fish	CAST	DES	DES3	PKCS#1 OAEP	PKCS#1 v1.5	XOR	Unenc-rypted	Standard Deviation
Monobit Test	0.3563	0.8495	0.4129	0.7034	0.7001	0.6347	0.6617	0.0000	0.0000	0.0000	0.0000	0.3232
Block Frequency Test	0.6120	0.6700	0.8508	0.8717	0.6739	0.4795	0.0982	0.7956	0.6285	0.0000	1.0000	0.2973
Independent Runs Test	0.2928	0.2092	0.5690	0.9011	0.1275	0.5922	0.0351	0.0000	0.0000	0.0000	0.0000	0.2950
Longest Runs Test	0.3735	0.5508	0.2576	0.2018	0.3929	0.5349	0.4236	0.5016	0.2965	0.0000	0.0000	0.1845
Spectral Test	0.9641	0.9768	0.7489	0.3135	0.9367	0.5588	0.3628	0.1748	0.9982	0.0000	0.0000	**0.3791**
Non Overlapping Patterns Test	0.3824	0.8874	0.5719	0.7193	0.0479	0.7668	0.1335	0.7917	0.5233	0.0000	0.0000	0.3263
Overlapping Patterns Test	0.0123	0.0821	0.2920	0.3004	0.1922	0.8939	0.2518	0.1550	0.1712	0.0000	0.0000	0.2398
Universal Test	0.3101	0.2031	0.7855	0.4042	0.8567	0.8574	0.8058	0.3027	0.5298	0.0000	0.0000	0.3141
Serial Test	0.2937	0.1751	0.6898	0.8080	0.3829	0.6550	0.0151	0.5689	0.0489	0.0000	0.0000	0.2931
Approximate Entropy Test	0.3202	0.0553	0.5882	0.5285	0.2862	0.0851	0.3340	0.2450	0.1651	0.0000	0.0000	0.1909
Cumulative Sums Test	0.3824	0.9320	0.5635	0.6605	0.8950	0.3800	0.6888	0.0000	0.0000	0.0000	0.0000	0.3498
Random Excursions Test	0.8251	0.4261	0.4576	0.9930	0.8197	0.3436	0.4022	0.2735	0.5566	0.7000	0.7000	0.2208
Random Excursions Variant Test	0.6991	0.8571	0.3518	0.2654	0.8173	0.2099	0.3670	0.0811	0.4618	1.0000	0.3865	0.2839
Matrix Rank Test	0.5624	0.9083	0.5359	0.9268	0.7473	0.5678	0.3579	0.1282	0.8707	0.0000	0.0000	0.3323
Linear Complexity Test	0.9053	0.8132	0.8263	0.2563	0.7104	0.0709	0.8156	0.2682	0.8527	0.2072	0.3292	0.3047

Table 4: P-values of NIST Statistical Test Suite [6] tests across different ciphers and generators, reproduced from Sengupta et al. [31]. The shaded row marks the spectral test, whose standard deviation across ciphers is the highest reported and which motivates the use of frequency-domain features for encrypted traffic classification in [31].

Metric / Generator	AES	Blowfish	CAST	DES	DES3	ARC2	ARC4	ChaCha20	Salsa20	Mersenne	QRNG	XOR	Plaintext
Uniformity	0.8876	0.9006	0.0652	0.4866	0.7258	0.5301	0.3554	0.7578	0.1564	0.8801	0.7238	0.0000	0.0000
Proportion	0.9900	0.9900	0.9880	0.9860	0.9930	0.9880	0.9900	0.9840	0.9840	0.9860	0.9840	0.0000	0.0000

Table 5: Proportion of tests passed and p-value of the uniformity test for the spectral test for various generators. Green cells meet the respective criterion, red cells fail it: a uniformity p-value of at least 10^{-4} and a pass proportion of at least 0.9806 ($m = 1000$, cf. Table 2) [6].

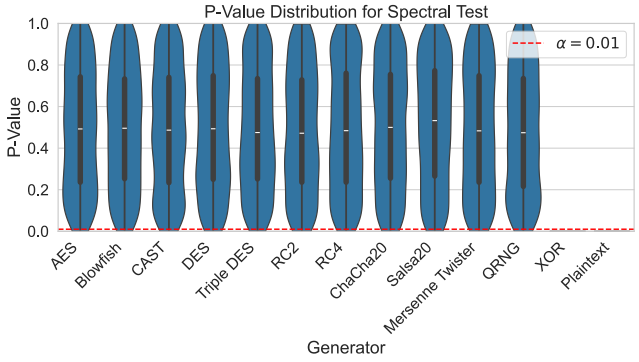


Figure 6: Violin plot of the spectral test p-values over different ciphers.

in Figure 6 are consistent with the expected behavior under the null hypothesis. As Table 5 reports, the p-value distributions appear approximately uniform and the test-pass proportions are close to the expected level for $\alpha = 0.01$ [6]. As expected, the XOR and plaintext baselines fail strongly because the construction introduces deterministic repetition when encrypting a zero-entropy stream with a fixed key.

These observations directly contradict the interpretation in Sengupta et al. [31] that the Spectral Test exhibits exceptional discriminative power across ciphers. Reporting a single p-value per cipher, as reproduced in Table 4, does not constitute adequate evidence for cipher-specific randomness claims, for the reasons given in Section 5.1.1. In our reproduction, cipher outputs are statistically indistinguishable from other high-entropy generators such as QRNG, which aligns with the broader critique that ciphertext-only traffic classifiers often rely on non-randomness artifacts or side channels rather than cryptographic "randomness differences" [22, 40].

Given that statistical test suites do not provide stable evidence for cipher separability in this setting, we next examine whether Sengupta et al.'s discriminative performance may instead be explained by properties of their DFT-based feature construction and evaluation protocol.

5.2 Discrete Fourier Transform Features

Sengupta et al. [31] propose frequency-domain features derived from the discrete Fourier transform (DFT) of packet-level bit sequences. For each packet, the bit stream is transformed into the frequency domain, yielding a complex-valued spectrum. Features are then computed on consecutive packet pairs (p_i, p_{i+1}) to capture the intuition that adjacent packets within the same flow may have similar plaintext structure, e.g. consecutive video frames, and that encryption may preserve exploitable relationships in the ciphertext representation. Concretely, the authors compute the Pearson correlation coefficient (PCC) and the Kullback–Leibler divergence (KLD) between the two spectra, and additionally compute moving averages and moving variances over the resulting PCC and KLD sequences (PCCA, PCCV, KLDA and KLDV).

A technical ambiguity in the original description is that PCC and KLD are not directly defined on complex-valued fast Fourier transform outputs. Moreover, KLD requires two valid probability

distributions. Since Sengupta et al. [31] do not specify the conversion, we implement the most natural interpretation, i.e. we map the complex FFT output to a non-negative real-valued representation using the magnitude spectrum and normalize it to sum to one, yielding a spectral density that can be treated as a discrete probability distribution for KLD. This choice is consistent with standard uses of frequency-domain "energy" representations and ensures that KLD is well-defined.

We implement the feature extraction pipeline on the offline sessions generated for the cipher randomness experiments. In doing so, one additional ambiguity arises: the paper does not specify whether features are computed independently per session or on a single long sequence obtained by concatenating all packets. These two choices are materially different because the moving average and variance features propagate information across packet boundaries. We therefore consider the following interpretations:

(1) **Session-local features:** Compute features independently within each session, i.e., reset moving statistics at the start of each session.
(2) **Globally concatenated features:** Concatenate all packets into one long sequence and compute features across the full stream.

In an offline dataset, packet order across sessions is not semantically meaningful, so the globally concatenated interpretation can introduce arbitrary cross-session dependencies. For the validation experiments below, we therefore adopt the session-local computation to ensure that features depend only on packets within the same session.

Figure 7 shows the empirical CDFs of the resulting DFT-based features across different ciphers. In contrast to the separations suggested by Sengupta et al. [31], the feature distributions overlap almost perfectly in our reproduction, indicating that these features do not provide stable cipher-specific separation under this controlled offline setting.

Despite the near-identical feature distributions in Figure 7, Sengupta et al. [31] report high classification accuracy for cipher and key-size identification using a random forest and 10-fold cross validation. We hypothesize that at least part of the reported performance can arise from temporal and structural leakage. This leak is not caused by plaintext data present in the dataset but arises by construction: features are computed on consecutive packet pairs, and the moving-average / moving-variance statistics couple neighboring packets within a session, so the features are not constructed independently but depend on knowing which packets are close to each other. Randomly splitting individual packets across folds can then place highly dependent samples in both training and test partitions, inflating the reported performance. We demonstrate this mechanism on our reconstruction and cannot verify that it occurred in the original setup, as the train / test split is not described in [31].

5.2.1 Cipher Classification. We compute the DFT-based features session-locally and then perform 10-fold cross validation with a decision tree classifier using default parameters in scikit-learn. We substitute a single decision tree for the random forest used by Sengupta et al. [31] because the random forest is prohibitively memory-intensive for our packet-level dataset at this scale. This substitution is a technical limitation arising from the volume of data we train on, and it does not understate the achievable performance. A genuine cipher-specific signal should survive a leak-resistant split even with a weaker classifier, so the point of the comparison is that performance collapses when the leak is removed, not that a stronger classifier could not have done better in the presence of the leak.

We evaluate cipher classification under two cross-validation strategies that differ only in whether they prevent cross-fold dependencies between packets from the same session:

(1) **Leak-prone split (packet-level):** All packets are randomly split across folds, allowing packets from the same session to appear in both training and test folds.
(2) **Leak-resistant split (session-level):** All packets belonging to the same session are assigned to the same fold, ensuring that no session contributes to both training and test data.

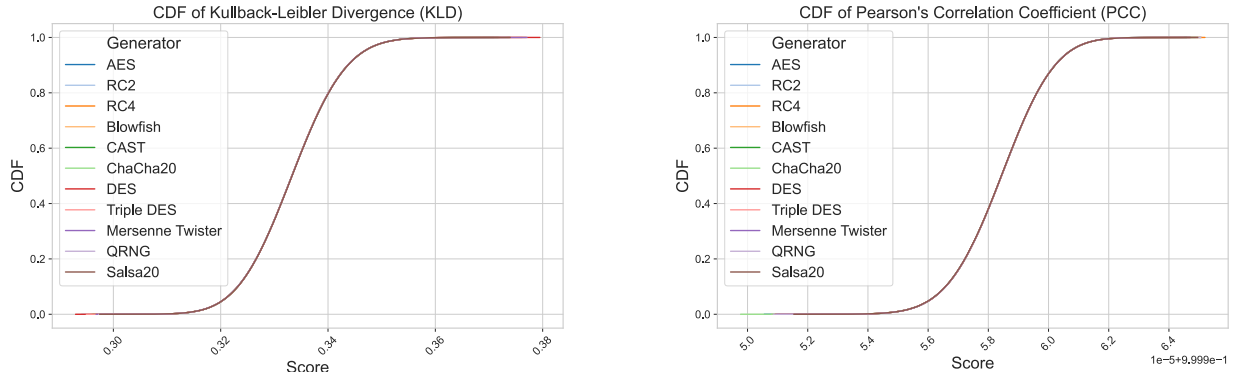
Macro-averaged over the balanced classes, precision, recall, and F1-score are 0.20 for the leak-prone split and 0.09 for the leak-resistant split, in line with the aggregation mode recommended by Zhao et al. [44]. Per-class results for all ciphers, averaged across the folds, are reported in Table 7 in the appendix.

5.2.2 Discussion. To avoid artificially inflating the metrics with trivially non-random baselines, plaintext and XOR are excluded from this classification experiment. Under the leak-prone packet-level split, the mean cross-validated accuracy is 0.2058 ± 0.0003 across folds. Under the leak-resistant session-level split, accuracy drops to 0.0908 ± 0.0005, which matches random guessing for 11 classes (≈ 0.0909). This controlled comparison supports the hypothesis that the DFT-based feature pipeline can exhibit strong apparent performance when the evaluation protocol allows structurally dependent samples to be split across training and test sets.

In the network-traffic setting discussed by Sengupta et al. [31], the authors propose computing features on consecutive packets within flows and suggest handling ambiguous attribution by maintaining a set of "recent reference packets" from candidate applications. However, the operational details of this mechanism and its interaction with evaluation splits remain unclear.

In particular, without explicitly enforcing session-level separation between training and test partitions, adjacent-packet feature construction and moving-statistic aggregation can again introduce leakage channels that lead to overly optimistic results. Whether such leakage occurred in the original evaluation cannot be determined, since the train / test split is not described in [31].

Takeaway. Three shortcomings limit the evidence in [31]. First, a single p-value per cipher is reported, which is one draw from a uniform distribution under the null hypothesis and therefore carries no discriminative information, whereas NIST SP 800-22 evaluates pass proportions and p-value uniformity across many sequences. Second, in a multi-sequence evaluation the DFT-based features that motivate the method show no separation between ciphers, with the feature distributions overlapping almost perfectly. Third, the feature construction couples adjacent packets through its averaging step, so an evaluation that splits packets rather than sessions can report performance that a leak-resistant split does not sustain. The first two are properties of the statistical evidence itself. The third



(a) KLD (b) PCC

Figure 7: CDFs of the DFT features for different ciphers. Only a single curve is visible because the curves overlap almost perfectly. Plots for moving average and variance are shown in the appendix (Figure 10).

is a mechanism we demonstrate on our reconstruction and cannot confirm for the original evaluation.

6 Reproducing Kanda and Hashimoto (2021)

Kanda and Hashimoto [17] propose a feature engineering approach based on randomness-test statistics to distinguish TLS cipher suites using only encrypted TLS application data, reporting an accuracy of 0.893 for identifying the encryption algorithm and 0.677 for identifying it together with key length. Unlike the deep learning-based classifiers that dominate this literature, their approach makes an explicit, testable claim grounded in classical statistics, making it straightforward to isolate where the reasoning goes wrong. We include it in our replication study for two reasons. First, it directly operationalizes the foundational assumption of Sengupta et al. [31] — that ciphertexts from different cipher suites exhibit statistically distinguishable non-randomness — allowing us to trace concretely how a flawed premise compounds into downstream methodology and reported results. Second, such works do not remain isolated: they are absorbed into surveys [41] and cited as independent corroboration of a claim the original evidence does not support.

6.1 Monobit and Runs Features

Kanda and Hashimoto [17] begin by selecting NIST STS tests with short minimum-length requirements, including the monobit test, frequency within a block, runs, longest run of ones in a block, serial, approximate entropy, and cumulative sums. For each test they consider both the p-value and additional features derived from intermediate quantities produced by the test. Based on feature-importance analyses, they report that features derived from the monobit and runs tests yield the strongest classification performance. Concretely, the reported feature set is:

- **Monobit:** $\frac{S_{0n}}{n}$, $\frac{S_{1n}}{n}$, $\frac{S_{0n}}{\sqrt{n}}$, $\frac{S_{1n}}{\sqrt{n}}$, $S_{\text{obs}} = \frac{|S_{0n} - S_{1n}|}{\sqrt{n}}$, and the p-value
- **Runs:** $\frac{V_n(\text{obs})}{\sqrt{n}}$ and the p-value

where n is the bit-string length, S_{0n} and S_{1n} denote the number of zeros and ones, and $V_n(\text{obs})$ denotes the number of runs in the sequence. The three most influential features reported by the authors are $\frac{V_n(\text{obs})}{\sqrt{n}}$, $\frac{S_{0n}}{\sqrt{n}}$, and $\frac{S_{1n}}{\sqrt{n}}$.

Given the results in Section 4, which show no stable cipher-specific structure in NIST STS p-value distributions for well-designed ciphers, it is notable that the p-values themselves are not among the most important features. This motivates a closer look at the dominant terms. Under the null hypothesis of an IID unbiased Bernoulli bit stream, we have $\mathbb{E}[S_{0n}] = \mathbb{E}[S_{1n}] = \frac{n}{2}$, implying

$$\mathbb{E}\left[\frac{S_{0n}}{\sqrt{n}}\right] = \mathbb{E}\left[\frac{S_{1n}}{\sqrt{n}}\right] = \frac{\sqrt{n}}{2}.$$

Similarly, the expected number of runs in a Bernoulli sequence with success probability p is $2(n-1)p(1-p)+1$, which for $p = \frac{1}{2}$ gives $\mathbb{E}[V_n(\text{obs})] = \frac{n+1}{2}$, and therefore

$$\mathbb{E}\left[\frac{V_n(\text{obs})}{\sqrt{n}}\right] \approx \frac{\sqrt{n}}{2} \quad \text{for large } n.$$

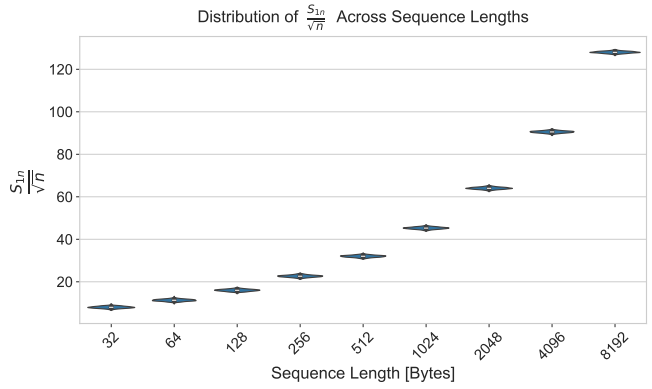
All three dominant features therefore scale as $\Theta(\sqrt{n})$, meaning they can act as proxies for the sequence length. This aligns with the observation by Wickramasinghe et al. [40] that length-related artifacts are often the strongest signal available to "ciphertext-only" classifiers.

To verify that these features primarily encode length rather than cipher-specific structure, we compute $\frac{S_{1n}}{\sqrt{n}}$ and $\frac{V_n(\text{obs})}{\sqrt{n}}$ on sequences drawn from a QRNG source at multiple lengths. Figure 8 shows that the feature distributions separate cleanly by n and are centered near the theoretical expectations for random data. This confirms that the dominant features are strongly length-dependent.

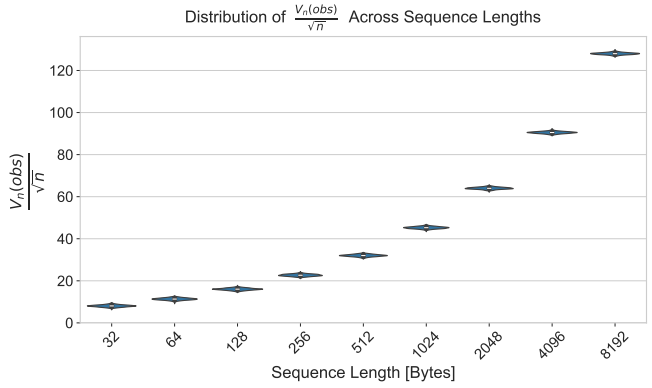
Taken together, these observations suggest that strong performance on real TLS traces would require that the amount of extracted TLS application data or the concatenated session length correlates with the negotiated cipher suite, either directly through cryptographic overheads or indirectly through experimental design.

6.2 Network Data Experiments

Kanda and Hashimoto [17] generate their dataset by collecting traffic from three HTTPS clients connecting to servers configured



(a) $\frac{S_{1n}}{\sqrt{n}}$



(b) $\frac{V_n(\text{obs})}{\sqrt{n}}$

Figure 8: Monobit- and runs-derived features on QRNG data for different sequence lengths.

with 13 different cipher suites, repeated 1000 times per client. They extract TLS application data and concatenate it per session to form the bit sequences used for feature computation. No fixed upper bound is imposed on session length, except that sequences are truncated to 6271 bits for the "Longest Run of Ones in a Block" test to keep parameters constant.

Since the dominant features above scale with $\sqrt{n}$, we investigate whether the session length n correlates with the cipher suite in a reproduction using the controlled TLS test range introduced in Section 3.1.2. Instead of fetching fixed-size blobs, the collector requests the webroot 1000 times from each server, mirroring the setup described by Kanda and Hashimoto [17]. From the resulting PCAPs we extract TLS application data records and group packets into sessions using `tcp.stream`. For each session we compute the total length of the extracted TLS application data. Table 6 summarizes the observed session lengths per server configuration.

Two patterns are immediately visible. First, within each fixed server configuration the session length is stable across the 1000 connections. For the TLS 1.2 suites every observed session has exactly the length reported in Table 6, while the TLS 1.3 configurations show only minor variation around the reported value, so Table 6 lists the median in those cases. This stability is expected when repeatedly requesting the same resource. Second, TLS 1.3 sessions are substantially longer than TLS 1.2 sessions because TLS 1.3 encrypts most handshake messages early and transports them inside application-data records [29].

TLS	Cipher Suite	Length [Bytes]
1.3	TLS_AES_256_GCM_SHA384	2568
1.3	TLS_CHACHA20_POLY1305_SHA256	2526
1.3	TLS_AES_128_GCM_SHA256	2504
1.2	(EC)DHE-RSA-CHACHA20-POLY1305	459 / 457
1.2	(EC)DHE-RSA-AES256-GCM-SHA384	475 / 473
1.2	(EC)DHE-RSA-AES128-GCM-SHA256	475 / 473
1.2	(EC)DHE-RSA-AES128-SHA256	538 / 538
1.2	ECDHE-RSA-AES256-SHA384	570
1.2	DHE-RSA-AES256-SHA256	538

Table 6: Session length per cipher suite and TLS version. (EC)DHE rows report ECDHE / DHE lengths. TLS 1.2 lengths are constant across the 1000 connections. For TLS 1.3 the median is reported.

Within TLS 1.2, session length differences are also systematically related to suite construction and integrity overhead. For example, `ECDHE-RSA-AES256-SHA384` and `ECDHE-RSA-AES128-SHA256` differ primarily in the HMAC hash size used by the MAC-then-encrypt construction, which affects record expansion. In our traces, the total session length differs by 32 bytes, which is consistent with accumulating per-direction overhead differences across the request and response. In contrast, the GCM suites use AEAD with a fixed-size authentication tag, and we do not observe comparable length differences driven by the hash identifier in the suite name.

This length structure aligns closely with the confusion matrix reported by Kanda and Hashimoto [17] (reproduced in Figure 11 in the appendix) as misclassifications concentrate among cipher suites that produce similar session lengths. For instance, the four AEAD-GCM suites are nearly indistinguishable by length:

- `ECDHE-RSA-AES256-GCM-SHA384` (475 bytes)
- `ECDHE-RSA-AES128-GCM-SHA256` (475 bytes)
- `DHE-RSA-AES256-GCM-SHA384` (473 bytes)
- `DHE-RSA-AES128-GCM-SHA256` (473 bytes)

Classification performance in such groups is expected to approach random guessing when the dominant signal is n.

Repeatedly fetching a fixed resource from each server creates unusually stable session lengths that amplify this confound. Since the original traces are not available, we demonstrate this dependence on a faithful reconstruction of the setup rather than verifying that it produced the reported results.

Takeaway. The features reported as most important in [17] are analytically length proxies. All three scale as $\Theta(\sqrt{n})$ under the null hypothesis, so they encode the length of the tested sequence rather than any cipher-specific property, and we confirm this directly on QRNG data at multiple lengths. In the replicated setting, session length is in turn determined by the negotiated cipher suite through record framing and integrity overhead, and separates TLS 1.2 from TLS 1.3 almost perfectly. The reported classification performance is therefore consistent with a method that recovers protocol and

suite information from length alone, which would not be expected to generalize to traffic where payload sizes vary.

7 Conclusion

A recurring claim in the encrypted traffic classification literature is that differences in cipher randomness explain strong classification performance. Our paper challenges this premise. Across both offline ciphertext and network-collected TLS application data, our results indeed indicate that modern ciphers and cipher suites do *not* produce stable, discriminative deviations from ideal randomness when tests are interpreted according to the intended statistical workflow, including p-value distribution checks and pass-rate criteria. This negative statistical result is the core of our contribution and does not depend on the original datasets, as it concerns what the NIST STS can detect in ciphertext at all. Separately, we identify two mechanisms that account for how the reported numbers could arise, namely leakage introduced by the feature construction of [31] and the length dependence of the features used in [17]. Because the original datasets and several setup details are not available, these are demonstrated on faithful reconstructions and should be read as plausible explanations rather than as verified causes. Hence, our study underscores the importance of rigorous statistical reasoning and independent validation in network measurement research.

References

[1] Giuseppe Aceto, Domenico Ciuonzo, Antonio Montieri, and Antonio Pescapé. 2019. Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. *IEEE Transactions on Network and Service Management* 16, 2 (2019), 445–458.

[2] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. 2018. Brotli: A general-purpose data compressor. *ACM Transactions on Information Systems* 37, 1 (2018), 1–30.

[3] Mohammed M. Alani. 2010. Testing randomness in ciphertext of block-ciphers using DieHard tests. *International Journal of Computer Science and Network Security* 10, 4 (2010), 53–57.

[4] Blake Anderson and David McGrew. 2019. TLS beyond the browser: Combining end host and network data to understand application behavior. In *Proceedings of the Internet Measurement Conference*. 379–392.

[5] Vaibhav Bajpai, Mirja Kühlewind, Jörg Ott, Jürgen Schönwälder, Anna Sperotto, and Brian Trammell. 2017. Challenges with reproducibility. In *Proceedings of the Reproducibility Workshop*. 1–4.

[6] Lawrence Bassham, Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, Nathanael Heckert, James Dray, and San Vo. 2010. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. Special Publication 800-22, Revision 1a. National Institute of Standards and Technology, Gaithersburg, MD. https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=906762

[7] Mihir Bellare, Anand Desai, Eron Jokipii, and Phillip Rogaway. 1997. A concrete security treatment of symmetric encryption. In *Proceedings 38th Annual Symposium on Foundations of Computer Science*. IEEE, 394–403.

[8] Robert G. Brown, Dirk Eddelbuettel, and David Bauer. 2018. Dieharder: A Random Number Test Suite. https://webhome.phy.duke.edu/~rgb/General/dieharder.php.

[9] Zhiyong Bu, Bin Zhou, Pengyu Cheng, Kecheng Zhang, and Zhen-Hua Ling. 2020. Encrypted network traffic classification using deep and parallel network-in-network models. *IEEE Access* 8 (2020), 132950–132959.

[10] Cloudflare. 2025. Cloudflare Radar. https://radar.cloudflare.com/adoption-and-usage?dateRange=52w. Accessed: 2025-08-28.

[11] Ali Doğanaksoy, Barış Ege, Onur Koçak, and Fatih Sulak. 2010. Cryptographic Randomness Testing of Block Ciphers and Hash Functions. Cryptology ePrint Archive, Paper 2010/564. https://eprint.iacr.org/2010/564

[12] Kevin P. Dyer, Scott E. Coull, Thomas Ristenpart, and Thomas Shrimpton. 2012. Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail. In *2012 IEEE Symposium on Security and Privacy*. IEEE, 332–346.

[13] ENISA. 2020. *Encrypted Traffic Analysis*. ENISA Report. European Union Agency for Cybersecurity. https://www.enisa.europa.eu/publications/encrypted-traffic-analysis

[14] Urs Gasser, Nancy Gertner, Jack Goldsmith, Susan Landau, Joseph Nye, David O'Brien, Matthew G. Olsen, Daphna Renan, Julian Sanchez, Bruce Schneier, Larry Schwartztol, and Jonathan Zittrain. 2016. *Don't panic: making progress on the "Going Dark" debate*. Technical Report. Berkman Center for Internet & Society, Harvard University.

[15] Jing Yan Haw, Syed M. Assad, Andrew M. Lance, Nelly H. Y. Ng, Vikram Sharma, Ping Koy Lam, and Thomas Symul. 2015. Maximization of Extractable Randomness in a Quantum Random-Number Generator. *Physical Review Applied* 3, 5 (May 2015), 054004. doi:10.1103/PhysRevApplied.3.054004

[16] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. 2014. A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 263–274.

[17] Atsushi Kanda and Masaki Hashimoto. 2021. Identification of TLS communications using randomness testing. In *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1099–1106.

[18] Vasilios Katos. 2005. A randomness test for block ciphers. *Appl. Math. Comput.* 162, 1 (2005), 29–35. doi:10.1016/j.amc.2003.12.122

[19] Hyunchul Kim, Kimberly C. Claffy, Marina Fomenkov, Dhiman Barman, Michalis Faloutsos, and KiYoung Lee. 2008. Internet traffic classification demystified: myths, caveats, and the best practices. In *Proceedings of the 2008 ACM CoNEXT Conference*. 1–12.

[20] Pierre L'Ecuyer and Richard Simard. 2007. TestU01: A C library for empirical testing of random number generators. *ACM Trans. Math. Software* 33, 4 (2007), 1–40.

[21] Ming Li and Paul M. B. Vitányi. 2008. *An Introduction to Kolmogorov Complexity and Its Applications* (3 ed.). Springer. doi:10.1007/978-0-387-49820-1

[22] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. 2022. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference 2022*. 633–642.

[23] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, and Mohammdsadegh Saberian. 2020. Deep Packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing* 24, 3 (2020), 1999–2012.

[24] Itsik Mantin and Adi Shamir. 2002. A Practical Attack on Broadcast RC4. In *Fast Software Encryption (FSE 2001) (Lecture Notes in Computer Science, Vol. 2355)*. Springer, 152–164. doi:10.1007/3-540-45473-X_13

[25] George Marsaglia and Wai Wan Tsang. 2002. Some difficult-to-pass tests of randomness. *Journal of Statistical Software* 7, 3 (2002), 1–9.

[26] Eva Papadogiannaki and Sotiris Ioannidis. 2021. A survey on encrypted network traffic analysis applications, techniques, and countermeasures. *Comput. Surveys* 54, 6 (2021), 1–35.

[27] Andrew Reed and Michael Kranch. 2017. Identifying HTTPS-protected Netflix videos in real-time. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*. 361–368.

[28] Stuart Reid. 2015. r4nd0m. https://github.com/StuartGordonReid/r4nd0m. Commit a34e148.

[29] Eric Rescorla. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. RFC Editor. https://www.rfc-editor.org/rfc/rfc8446

[30] Roei Schuster, Vitaly Shmatikov, and Eran Tromer. 2017. Beauty and the burst: Remote identification of encrypted video streams. In *26th USENIX Security Symposium (USENIX Security 17)*. 1357–1374.

[31] Satadal Sengupta, Niloy Ganguly, Pradipta De, and Sandip Chakraborty. 2019. Exploiting diversity in Android TLS implementations for mobile app traffic classification. In *The World Wide Web Conference*. 1657–1668.

[32] Zhaolei Shi, Nurbol Luktarhan, Yangyang Song, and Gaoqi Tian. 2023. BFCN: A novel classification method of encrypted traffic based on BERT and CNN. *Electronics* 12, 3 (2023), 516.

[33] Dmitry Shkarin. 2002. PPM: One step to practicality. In *Proceedings DCC 2002. Data Compression Conference*. IEEE, 202–211.

[34] Robin Sommer and Vern Paxson. 2010. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*. IEEE, 305–316.

[35] Juan Soto and Lawrence Bassham. 2000. *Randomness Testing of the Advanced Encryption Standard Finalist Candidates*. NISTIR 6483. National Institute of Standards and Technology.

[36] Marek Sýs, Zdenek Riha, Vashek Matyas, Kinga Marton, and Alin Suciu. 2015. On the interpretation of results from the NIST statistical test suite. *Romanian Journal of Information Science and Technology* 18, 1 (2015), 18–32.

[37] Erik van der Kouwe, Gernot Heiser, Dennis Andriesse, Herbert Bos, and Cristiano Giuffrida. 2019. SoK: Benchmarking Flaws in Systems Security. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. 310–325. doi:10.1109/EuroSP.2019.00031

[38] Thijs Van Ede, Riccardo Bortolameotti, Andrea Continella, Jingjing Ren, Daniel J. Dubois, Martina Lindorfer, David Choffnes, Maarten Van Steen, and Andreas Peter. 2020. Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. In *Network and Distributed System Security Symposium*.

[39] Xin Wang, Shuhui Chen, and Jinshu Su. 2020. App-net: A hybrid neural network for encrypted mobile traffic classification. In *IEEE INFOCOM 2020 - IEEE*

Conference on Computer Communications Workshops (INFOCOM WKSHPS). IEEE, 424–429.

[40] Nimesha Wickramasinghe, Arash Shaghaghi, Gene Tsudik, and Sanjay Jha. 2025. SoK: Decoding the Enigma of Encrypted Network Traffic Classifiers. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1825–1843. doi:10.1109/sp61157.2025.00165

[41] Sunghyun Yu and Yoojae Won. 2023. A survey of methods for encrypted network traffic fingerprinting. *Mathematical Biosciences and Engineering* 20, 2 (2023), 2183–2202. doi:10.3934/mbe.2023101

[42] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Fangqi Li, Yanhao Wang, Yijun Wang, Guan Gui, and Zhi Xue. 2024. A novel self-supervised framework based on masked autoencoder for traffic classification. *IEEE/ACM Transactions on Networking* 32, 3 (2024), 2012–2025.

[43] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Yanhao Wang, Yijun Wang, Guan Gui, and Zhi Xue. 2023. Yet Another Traffic Classifier: A Masked Autoencoder Based Traffic Transformer with Multi-Level Flow Representation. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 4 (June 2023), 5420–5427. doi:10.1609/aaai.v37i4.25674

[44] Yuqi Zhao, Giovanni Dettori, Matteo Boffa, Luca Vassio, and Marco Mellia. 2025. The Sweet Danger of Sugar: Debunking Representation Learning for Encrypted Traffic Classification. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) *(SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 296–310. doi:10.1145/3718958.3750498

A Ethics

This work does not raise any ethical issues.

B Online Resources

The source code and supplementary materials associated with this work are publicly available in a GitLab repository. The repository contains all relevant implementations, data processing scripts, and documentation required to reproduce the results presented in this study. It can be accessed at: https://gitlab.com/b1520dd103c25532/encrypted-traffic-randomness-myths

C TLS Range Deployment Details

The range is deployed on an isolated Docker network in 172.20.0.0/16. For containerization, we use Docker and deploy the range with Ansible to efficiently create the containers from Jinja templates. For server-side TLS version and cipher suite experiments, we use `nginx:mainline` (Debian 12) and pre-generate server configurations that fix the TLS version and cipher suite, using `ssl_protocols` together with `ssl_ciphers` for TLS 1.2 and below and `ssl_ciphersuites` for TLS 1.3. The collector is a custom image based on `alpine:edge` and includes `openssl` 3.5.1 built with support for HTTP/2, HTTP/3, TLS 1.0 to TLS 1.3 and QUIC, together with `nghttp2` 1.66.0, `nghttp3` 1.10.1, and `ngtcp2` 1.13.0. We use `curl` as the HTTP client, built with the required protocol features.

D Other Test Batteries

Dieharder includes all NIST STS tests as a subset, and TestU01 covers analogous statistical properties through its own test families. There are also practical obstacles to applying them here. Our packet-level sequences of 1504 bytes are too short for many of the more demanding tests in these batteries, which typically require input sizes on the order of 10^7 bits to achieve meaningful statistical power. Moreover, modern ciphers are designed to be indistinguishable from random under any efficient statistical test and are well documented to pass these batteries.

Cipher	Precision	Recall	F1-Score
AES	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
ARC2	0.21 / 0.09	0.21 / 0.09	0.21 / 0.09
ARC4	0.21 / 0.09	0.21 / 0.09	0.21 / 0.09
Blowfish	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
CAST	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
ChaCha20	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
DES	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
DES3	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
Mersenne	0.20 / 0.09	0.20 / 0.09	0.20 / 0.09
QRNG	0.21 / 0.09	0.20 / 0.09	0.21 / 0.09
Salsa20	0.20 / 0.09	0.21 / 0.09	0.20 / 0.09

Table 7: Per-class precision, recall and F1-score for the leak-prone (left) and leak-resistant (right) splits, averaged across the 10 cross-validation folds.

E NIST STS Implementation Notes

To make the NIST STS usable in an automated pipeline, we implemented a batch mode that exposes all required parameters via command-line arguments and removed the need for interactive input, and we adapted the output format to simplify parsing in post-processing. We also patched two robustness issues encountered during experimentation, namely an off-by-one error affecting the spectral test and a crash during report generation for shorter sequences. Finally, we compiled the suite with `-O3` using `gcc` to reduce runtime, noting that compiler optimization does not change numerical results for the evaluated configurations.

F Cipher Classification

In Section 5.2.1, the leakage hypothesis is tested with 10-fold cross validation under a leak-prone and a leak-resistant split. Table 7 reports the per-class metrics for both strategies.

G Compression-Based Randomness Testing Using General-Purpose Compressors

This section examines the use of general-purpose compressors as a proxy for algorithmic randomness testing. The underlying intuition is that highly random sequences should be largely incompressible, whereas structured or biased data may admit shorter encodings. While this idea is well established, its applicability to encrypted data is limited and requires careful interpretation. The analysis uses the same dataset employed for comparing structured generators and cryptographic ciphers. For each sequence, we compute compression-ratio features using the general-purpose compressors Gzip, Bzip2, LZMA, Zlib, LZ4, Zstd, Brotli, and PPMd. For compressors available in the Python standard library, we use the built-in implementations. Brotli [2] is a lossless compressor optimized for web content and combining dictionary compression with entropy coding and context modeling. PPMd (Prediction by Partial Matching, variant D) [33] is a statistical compressor that models symbol probabilities from preceding context. Brotli, zstd, and PPMd are accessed via the Python packages `brotli`, `pyzstd`, and `pyppmd`, respectively. As the feature value, we compute $r_{compression} = \frac{l_{original} - l_{compressed}}{l_{original}}$ for each sequence, where l denotes the byte length, which yields one compression-ratio feature per compressor.

Figure 9 excludes structured, plaintext and XOR-based data, as their compression ratios differ by several orders of magnitude

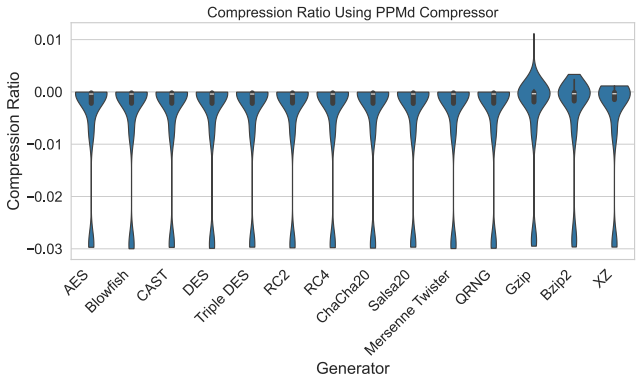


Figure 9: Violin plot of compression ratio using the PPMd compressor on compressed and ciphertext data.

from those of the ciphertext and compressor-generated sequences. Across all evaluated ciphers, the resulting compression-ratio distributions are effectively indistinguishable. In particular, no positive compression ratio is achieved for ciphertext data. The negative compression ratios observed are a consequence of the additional overhead introduced by the compression format, which outweighs any potential redundancy in the input.

For the outputs of general-purpose compressors, a fraction of sequences can be compressed further, while others remain incompressible. The proportion of compressible sequences varies across compressors, for example, Gzip compresses a noticeably larger subset of sequences than XZ. This difference is more pronounced than the variation observed between cipher outputs and is easier to distinguish than differences in p-value distributions obtained from statistical randomness tests.

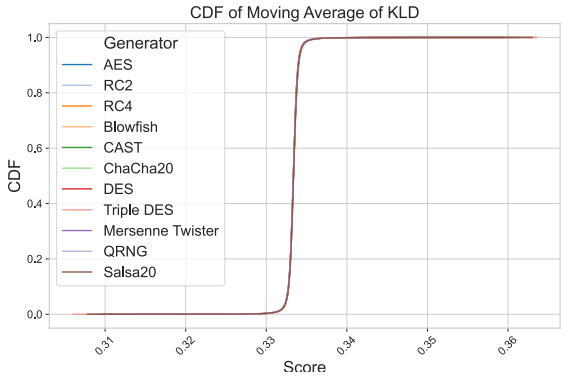
It should be noted that this analysis relies exclusively on general-purpose compression algorithms, which are not designed to exploit highly specific or subtle structure. A sequence that appears incompressible under such compressors may still admit a shorter description under a specialized encoding scheme. Consequently, compression-based measures provide at best an upper bound on algorithmic randomness and should be interpreted with caution when applied to cryptographic data.

H Moving Statistics of the DFT Features

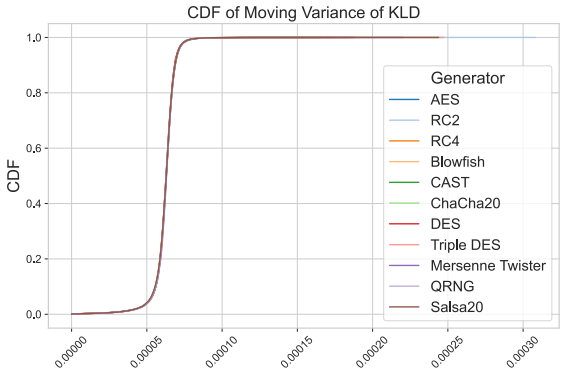
The plots in Figure 10 complement Figure 7 with the moving average and variance of the KLD and PCC features from Section 5.2.

I Confusion Matrix from Kanda and Hashimoto

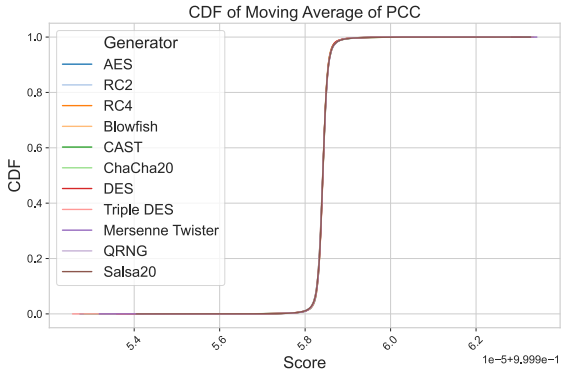
In Section 6.2, the confusion matrix from Kanda and Hashimoto [17] is referenced. We reproduce it in Figure 11.



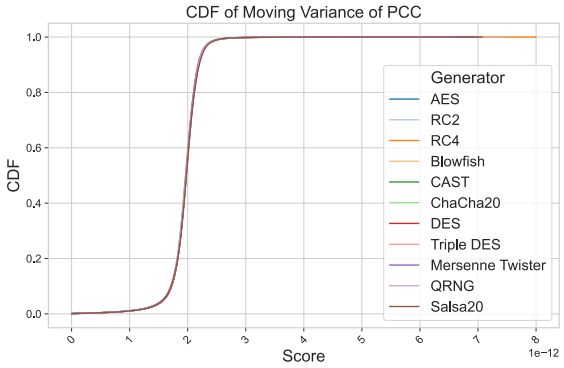
(a) Moving Avg. of KLD



(b) Moving Var. of KLD



(c) Moving Avg. of PCC



(d) Moving Var. of PCC

Figure 10: CDFs of the DFT features for different ciphers. Only a single curve is visible because the curves overlap almost perfectly.

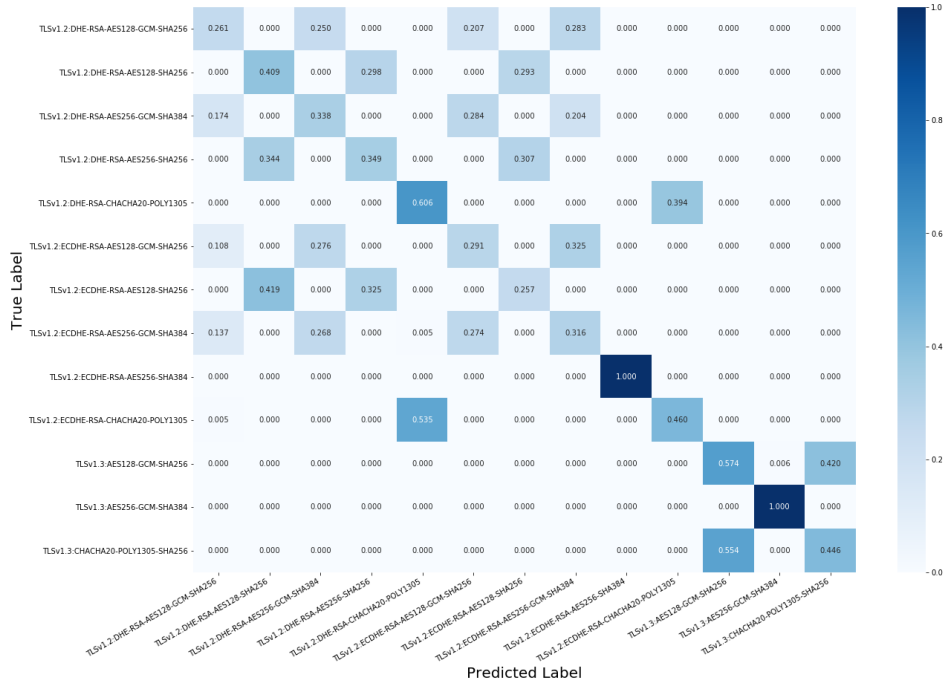


Figure 11: Confusion matrix for cipher suite classification based on session length, reproduced from Kanda and Hashimoto [17].